

Advanced RL Methods

Reinforcement learning – LM Artificial Intelligence
(2025-26)

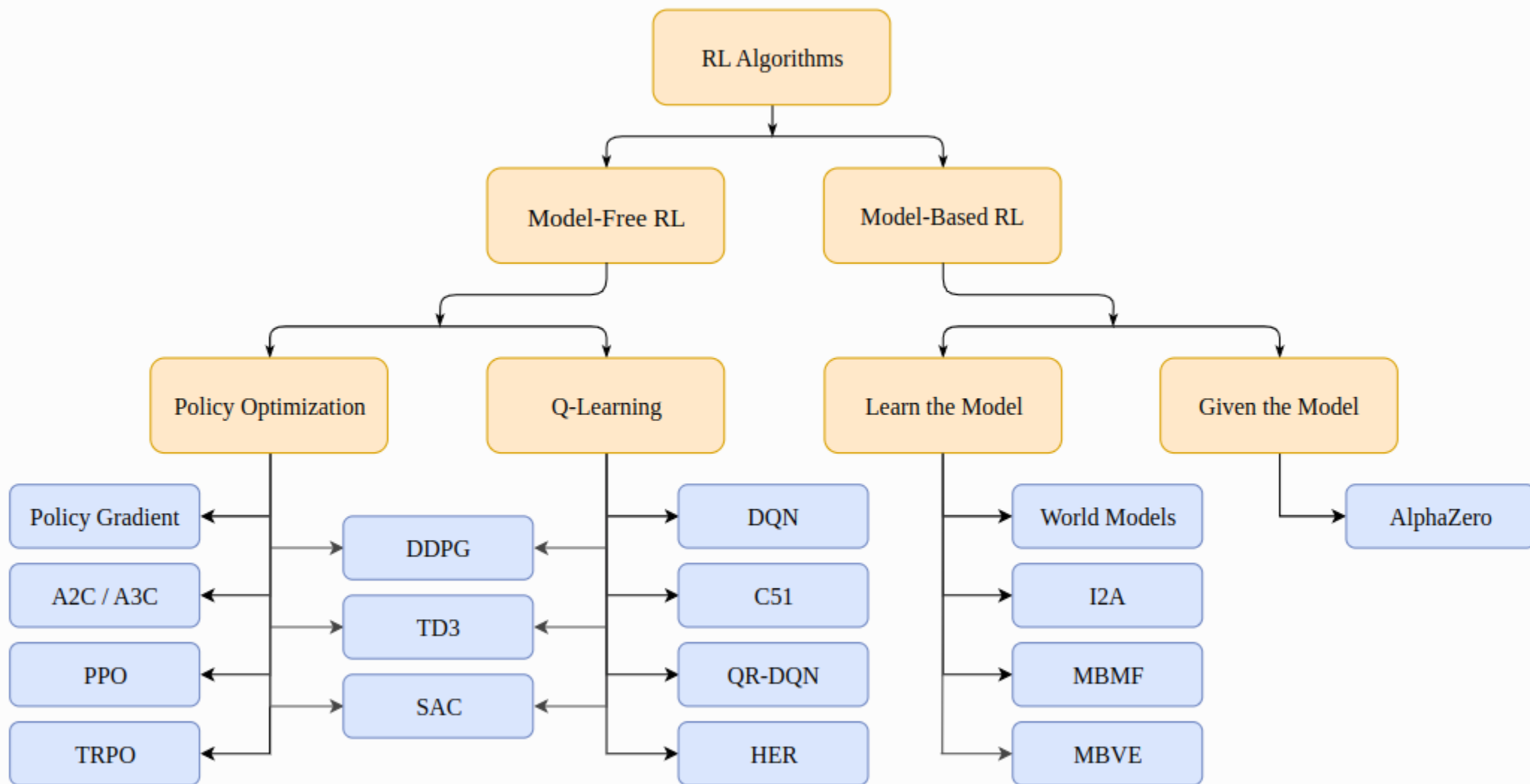
Alberto Castellini
University of Verona

Summary

- A Taxonomy of RL Algorithms
- Model-Free vs Model-Based RL
- What to learn
 - In Model-Free RL
 - In Model-Based RL
- Monte Carlo Tree Search
- AlphaGo

A Taxonomy of RL Algorithms

A Taxonomy of RL Algorithms



References

A2C / A3C (Asynchronous Advantage Actor-Critic): Mnih et al, 2016

PPO (Proximal Policy Optimization): Schulman et al, 2017

TRPO (Trust Region Policy Optimization): Schulman et al, 2015

DDPG (Deep Deterministic Policy Gradient): Lillicrap et al, 2015

TD3 (Twin Delayed DDPG): Fujimoto et al, 2018

SAC (Soft Actor-Critic): Haarnoja et al, 2018

DQN (Deep Q-Networks): Mnih et al, 2013

C51 (Categorical 51-Atom DQN): Bellemare et al, 2017

QR-DQN (Quantile Regression DQN): Dabney et al, 2017

HER (Hindsight Experience Replay): Andrychowicz et al, 2017

World Models: Ha and Schmidhuber, 2018

I2A (Imagination-Augmented Agents): Weber et al, 2017

MBMF (Model-Based RL with Model-Free Fine-Tuning): Nagabandi et al, 2017

MBVE (Model-Based Value Expansion): Feinberg et al, 2018

AlphaZero: Silver et al, 2017

Source: https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html

Model-Free vs Model-Based RL

Model-free vs Model-based RL

- **Important branching point** in RL algorithms: Does the agent have access to (or learns) a **model of the environment**?
- **Model of the environment**: function that predicts **state transitions** and/or **rewards**
- **Advantage** of having a **model**: it allows the agent to **plan** by **thinking ahead** (bridging **RL** and **planning**)
- **Consequences**: substantial **improvement** in **sample efficiency** over methods that don't have a model.
- **Downside**: a **ground-truth model** of the environment is usually **not available** to the agent.
 - The agent has to **learn** the model from experience → **challenges**

Model-free vs Model-based RL

- The **bias** in the **model** can be exploited by the agent: **the agent could perform well on the learned model, but sub-optimally in the real environment**
- **Model-learning is fundamentally hard**: even intense effort (e.g., time) can fail to pay off
- **Model-free methods**: **forego** potential gains in **sample efficiency** from using a model **but** they tend to be **easier** to implement and tune
- **Currently, model-free** methods are **more popular** and have been **more extensively developed and tested** than model-based methods

Examples of model-based algorithms and their applications:

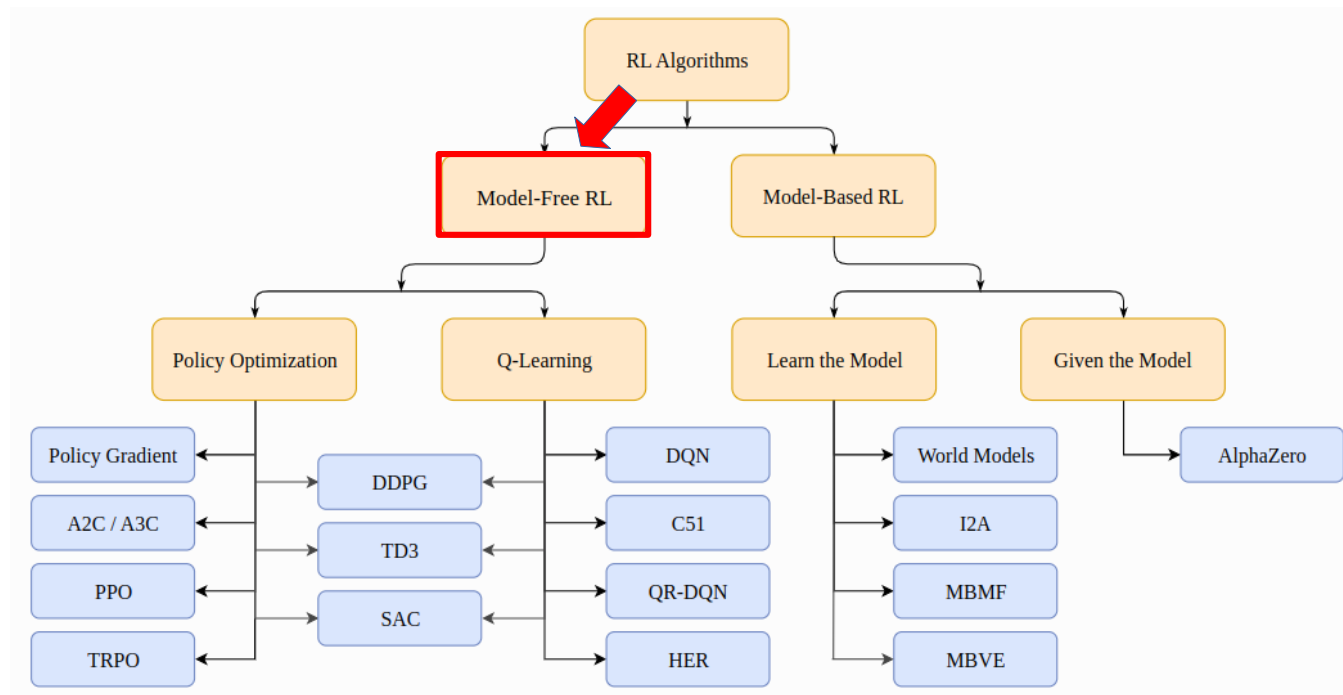
- **Dyna-Q** (Sec. 8.1-8.3 S&B)
 - Sutton, R. S. (1991). **Dyna, an integrated architecture for learning, planning, and reacting**. SIGART Bulletin, 2(4):160–163. ACM, New York.
- **Monte Carlo Tree Search** (Sec. 8.10, 8.11 S&B)
 - Browne C. B. et al. (2012). **A Survey of Monte Carlo Tree Search Methods**. IEEE Transactions on Computational Intelligence and AI in Games, 4(1):1-43.
 - Kocsis, L., Szepesvári, C. (2006). **Bandit Based Monte-Carlo Planning**. In: Machine Learning: ECML 2006. Lecture Notes in Computer Science, 4212. Springer, Berlin, Heidelberg.
- **AlphaGo and AlphaZero** (Sec 16.6 S&B) ← **Very popular**
 - Silver, D. et al. (2016). **Mastering the game of Go with deep neural networks and tree search**. Nature, 529(7587):484–489.
 - Silver, D. et al. (2017). **Mastering the game of Go without human knowledge**. Nature, 550 (7676):354–359.
 - Silver, D. et al. (2018). **A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play**. Science, 362 (6419):1140-1144.

What to learn

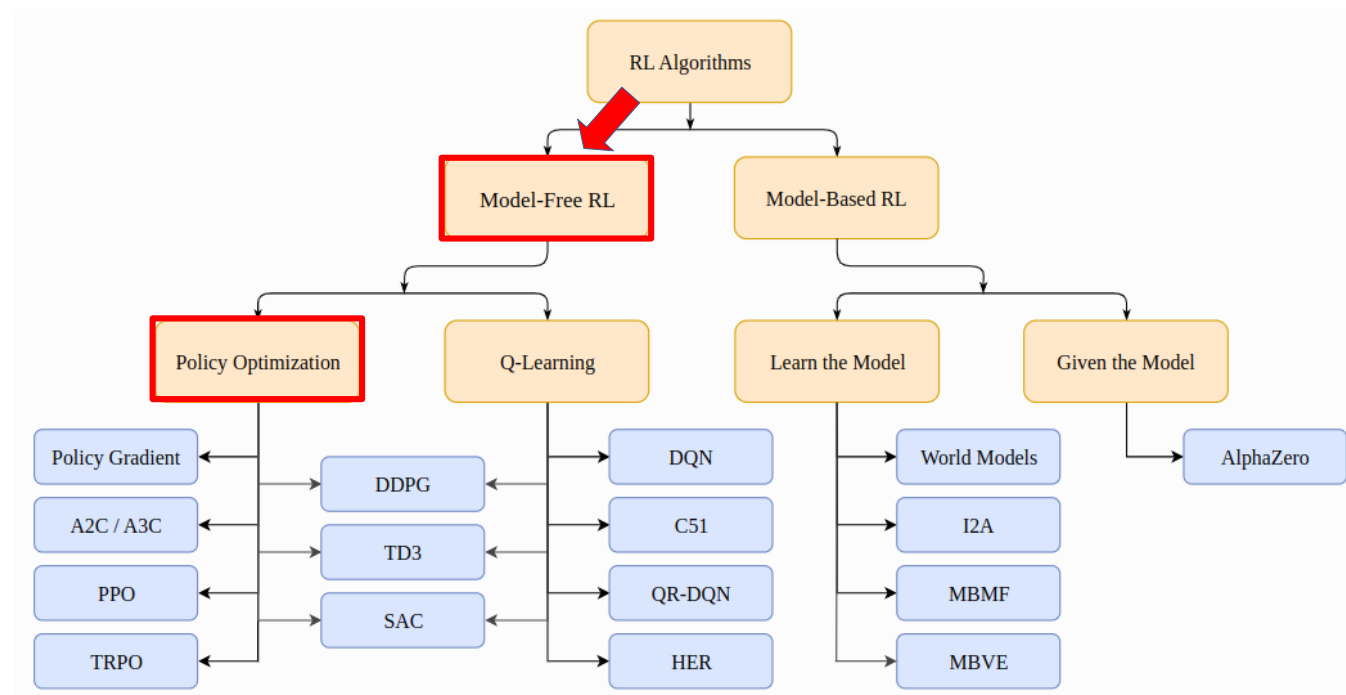
What to learn

- **Another critical branching point** in an RL algorithm is the question of **what to learn**
- 4 elements that can be learned are:
 - **Policies** (stochastic or deterministic)
 - **Action-value functions** (Q-functions)
 - **Value functions**
 - **Environment models**

What to learn in Model-free RL



What to learn in Model-Free RL: *Policy optimization*



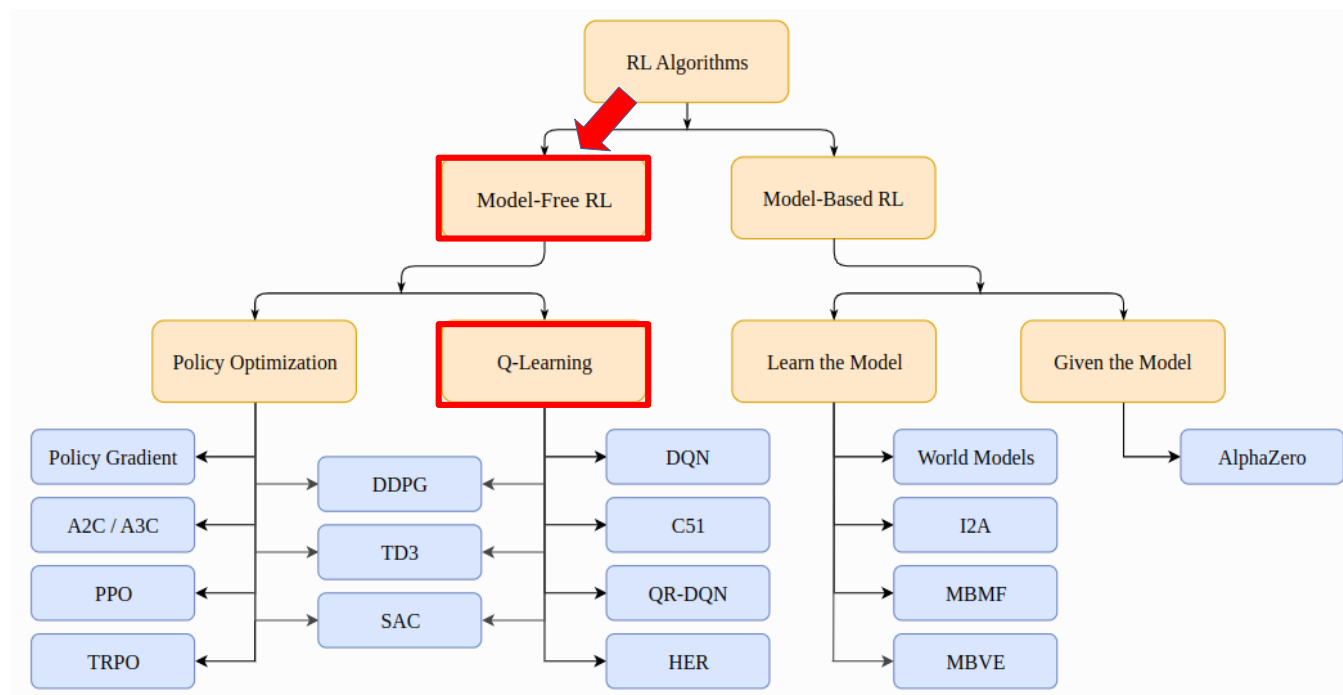
- Methods in the policy optimization family explicitly represent the **policy** and **learn its parameters** in two ways:
 - **Directly**: gradient ascent on the policy performance objective $J(\pi_\theta)$
 - **Indirectly**: by maximizing local approximations of $J(\pi_\theta)$
- The optimization is almost always performed **on-policy**
- These methods usually learn also an approximation of the **value function** $V^\pi(s)$ used to figure out how to update the policy

What to learn in Model-Free RL: Policy optimization

Examples of state-of-the-art **policy optimization** algorithms:

- **Advantage Actor Critic (A2C/A3C)**: performs **gradient ascent** to **directly** maximize performance
 - Mnih V. et al. (2016). **Asynchronous Methods for Deep Reinforcement Learning**. ArXiv:1602.01783
- **Proximal Policy Optimization (PPO)**: **indirectly** maximizes performance, by **maximizing a surrogate objective function** which gives a conservative **estimate** for how much $J(\pi_\theta)$ changes as a result of the update
 - Schulman J. et al. (2016). **Proximal Policy Optimization Algorithms**. ArXiv:1707.06347

What to learn in Model-Free RL: *Q-learning*



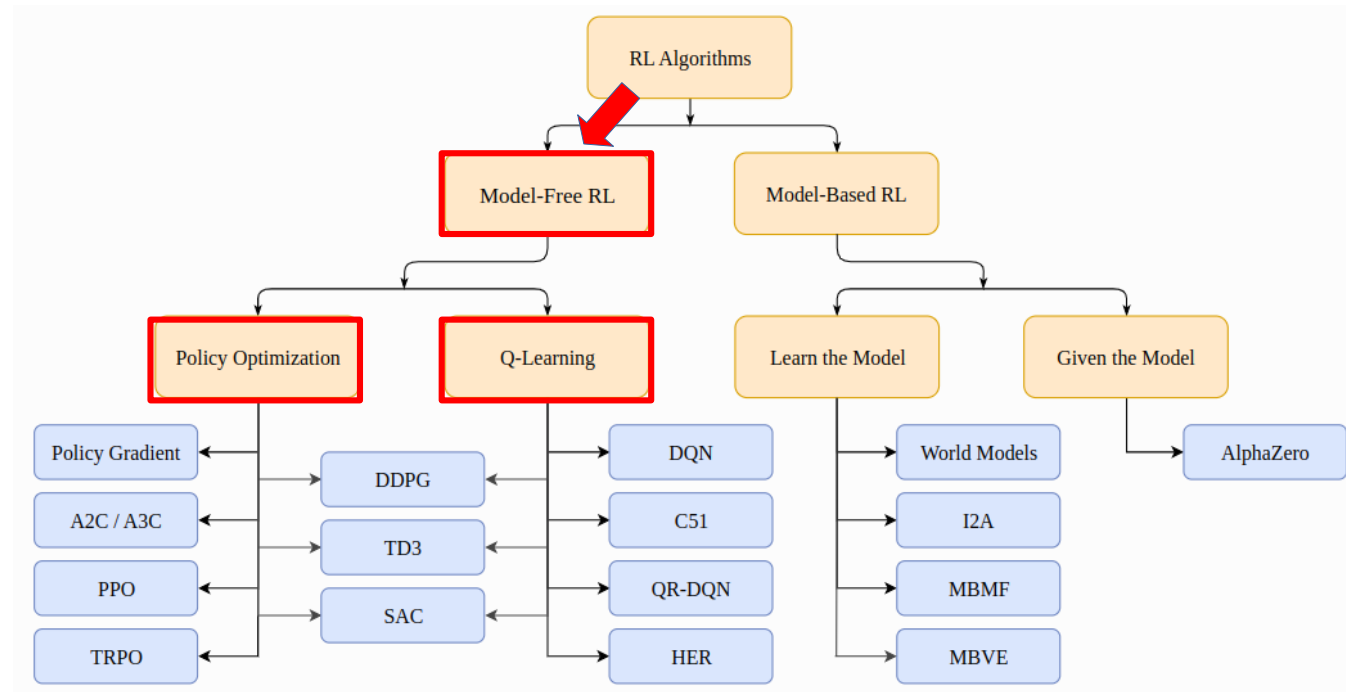
- Methods in this family **learn** an **approximation** $Q_{\theta}(s, a)$ for the **optimal action-value function**
- **Objective function**: typically based on the Bellman equation
- The optimization is almost always performed **off-policy**
- The **policy** is obtained by selecting actions having maximum Q-value

What to learn in Model-Free RL: Q-learning

Examples of state-of-the-art Q-learning based algorithms

- **Deep Q Networks (DQN)**: a classic which substantially launched the field of deep RL
 - Mnih, V. et al. (2013). **Playing atari with deep reinforcement learning**. ArXiv:1312.5602.
 - Mnih, V. et al. (2015). **Human-level control through deep reinforcement learning**. Nature, 518(7540):529-533.
- **C51**: a **variant** that learns a **distribution** over **return** whose expectation is the optimal Q-value
 - Bellemare M. G. et al. (2017). **A Distributional Perspective on Reinforcement Learning**. ArXiv:1312.5602.

What to learn in Model-Free RL: *Interpolation between Policy optimization and Q-learning*



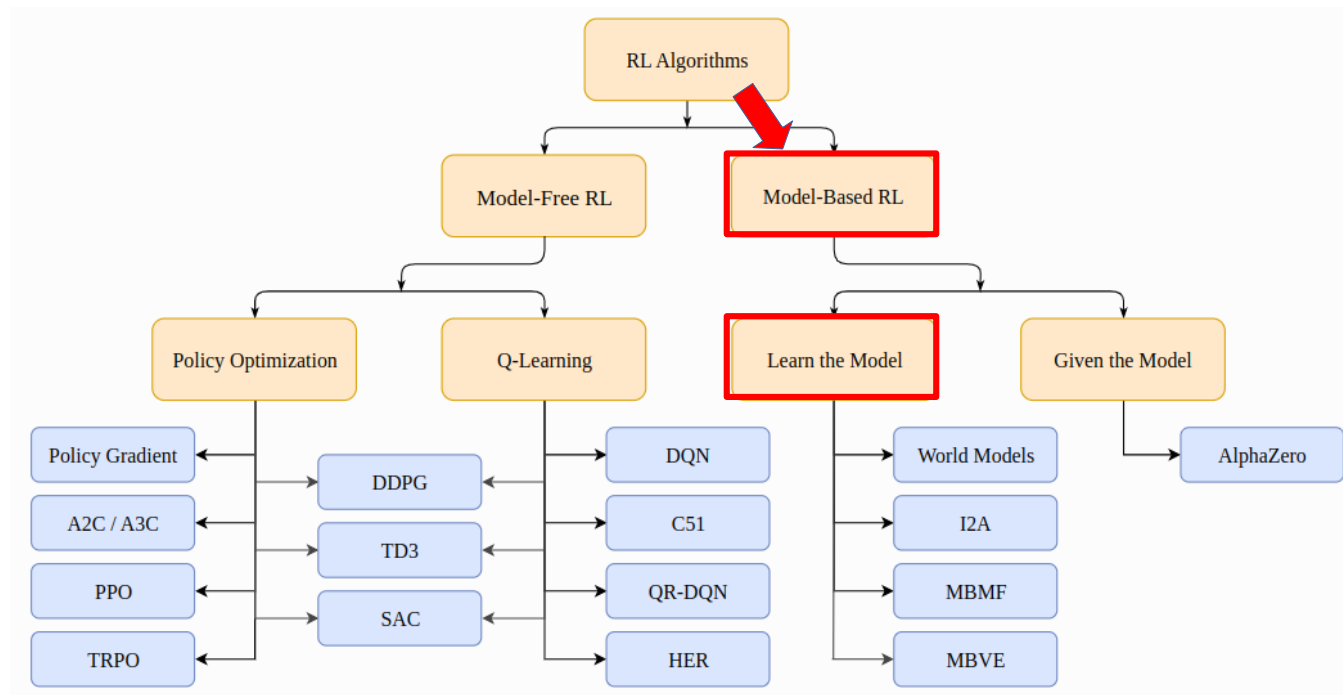
- **Policy optimization** methods are **stable** and **reliable**
- **Q-learning methods** are less stable but they are more **sample efficient** because they reuse data more effectively than policy optimization methods
- The two methods are **not incompatible**. There exist algorithms that live **in between** the two extremes. They **trade-off** between the **strengths** and **weaknesses** of the two families.

What to learn in Model-Free RL: Interpolation btw Policy optimization and Q-learning

Examples of state-of-the-art algorithms in this family:

- **Deep Deterministic Policy Gradient (DDPG)**: concurrently learns a **deterministic policy** and a **Q-function** by using each to improve the other
 - Lillicrap, T. P. et al. (2015). **Continuous control with deep reinforcement learning**. ArXiv:1509.02971.
- **Soft Actor-Critic (SAC)**: a variant which uses **stochastic policies**, **entropy regularization**, and a few **other tricks** to **stabilize learning** and **score higher** than DDPG on standard benchmarks.
 - Haarnoja A. et al. (2018) **Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor**. ICML 2018, <https://proceedings.mlr.press/v80/haarnoja18b/haarnoja18b.pdf>

What to learn in Model-based RL



What to learn in Model-based RL

- Unlike model-free RL algorithms, it is **not simple to group** model-based RL algorithms, since **there are many orthogonal ways of using models**
- We provide only **few examples** of model-based techniques. The list is far from exhaustive.

What to learn in Model-based RL: *Pure Planning*

- Most basic approach: **never explicitly represent the policy**. Use pure planning techniques like **Model-Predictive Control (MPC)** to select actions
- **Model Predictive Control (MPC)**: each time the agent **observes the environment**, it computes a **plan** which is **optimal** with respect to the **model**. The **plan** describes **all actions to take over some fixed window of time after the present**. The agent then **executes the first action of the plan**, and immediately **discards the rest** of it.
- **Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning (MBMF)**: explores **MPC** with **learned environment models** on some standard benchmark tasks for deep RL
 - Nagabandi, T. P. et al. (2017). **Continuous control with deep reinforcement learning**. ArXiv:1708.02596.

What to learn in Model-based RL: *Expert Iteration*

- Use and learn an **explicit representation of the policy**
- Main **steps**:
 - The agent uses the **model** in a **planning algorithm** (like Monte Carlo Tree Search)
 - Candidate **actions** are generated for the **plan** by sampling from its **current policy**
 - The **planning algorithm** produces an **action** which is **better** than **what the policy alone** would have produced (it is an “expert” relative to the policy)
 - The **policy** is afterwards **updated** to produce an action **more like the planning algorithm’s output**
- **AlphaZero**: uses this approach to train deep neural networks to chess and Shogi
 - Silver, D. et al. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. ArXiv:1712.01815.

What to learn in Model-based RL: *Data Augmentation for Model-Free Methods*

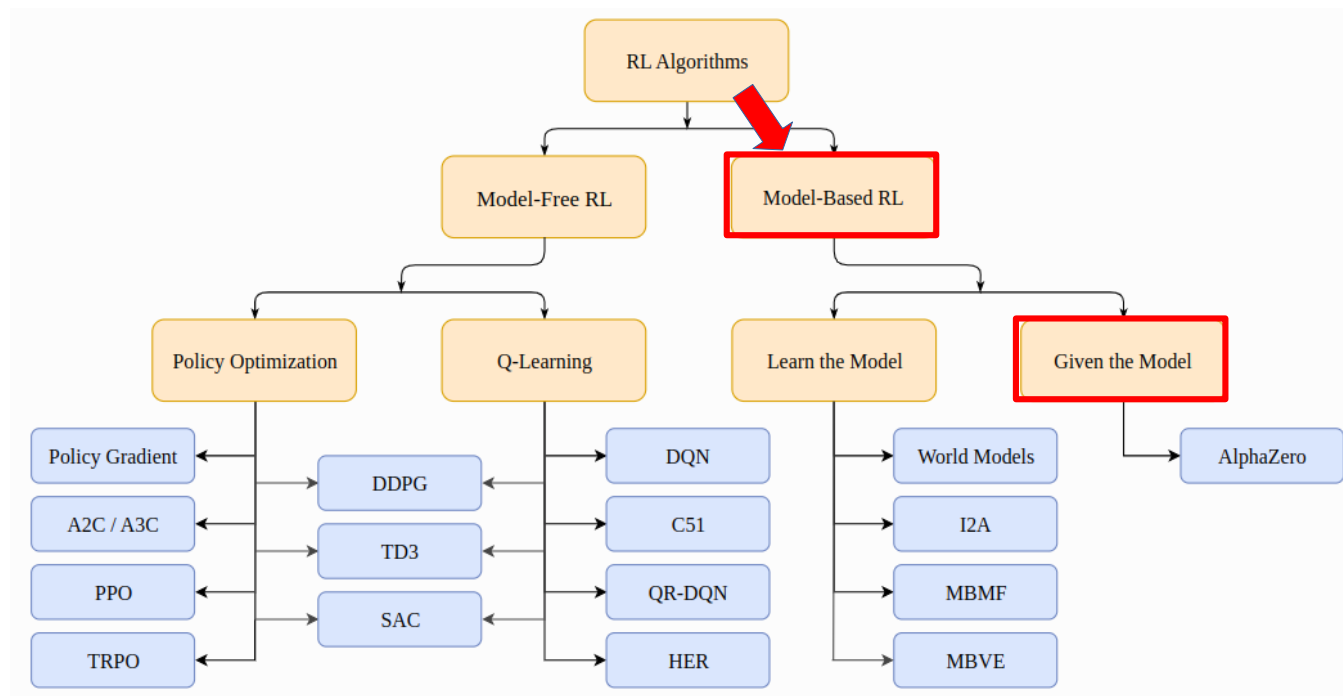
- Use a **model-free RL** algorithm to **train a policy or Q-function** but
 - **Augment real** experiences with **fictitious ones** in updating the agent **or**
 - **Use only fictitious experience** for updating the agent
- **Model-Based Value Estimation (MBVE)**: an example of **augmenting real** experiences with **fictitious** ones
 - Feinberg et al. (2018) **Model-Based Value Estimation for Efficient Model-Free Reinforcement Learning**. ArXiv:1708.02596
- **World Models**: an example of using **purely fictitious experience** to **train the agent**, which they call “training in the dream”
 - Ha et al., **Recurrent World Models Facilitate Policy Evolution**. Advances in Neural Information Processing Systems 31 (NeurIPS 2018)

What to learn in Model-based RL: *Embedding Planning loops into policies*

Idea:

- **Embed the planning procedure into a policy** as a subroutine (complete plans become side information for the policy)
- The policy is meanwhile trained with any standard **model-free** algorithm
- The policy **learns to choose how and when to use the plans** reducing the problem of model bias
- **I2A**: an example of agents being endowed with this style of imagination
 - Weber, T. et al. (2017). **Imagination-Augmented Agents for Deep Reinforcement Learning**. ArXiv:1707.06203.

What to learn in Model-based RL



Monte Carlo Tree Search (S&B Sec. 8.11)

Rollout Algorithms

- **Rollout algorithms:** **decision-time** planning algorithms based on **Monte-Carlo** control
- **Action values** are **estimated** by **averaging** the **returns** of **many simulated trajectories** that:
 - **Start** with **each possible action**
 - Then follow a **rollout policy**
- When action-value estimates are considered **accurate enough** the action with **highest value** is executed
- Then the process is **repeated** from the **next state**
- Tesauro, G., Galperin, G. R. (1997). **On-line policy improvement using Monte-Carlo search**. In Advances in Neural Information Processing Systems 9 (NIPS 1996), pp. 1068–1074. MIT Press, Cambridge, MA.

Rollout Algorithms

- Rollout algorithms are **online**, they estimate action values **only for the current state**. This allows **scaling to problems with large state space without using approximate value functions**
- Rollout algorithms **improve upon the rollout policy** (according to the policy improvement algorithm) but they do **not find the optimal policy**
- However, **in practice** they can be **very effective** and produce good performance **even** if the rollout **policy** is completely **random**
- The **better** the **rollout policy** and the **value estimate**, the **better** the **policy produced** → **Time/performance trade-off**
- **Parallelization** is possible since trajectories are independent
- **Truncations** of the simulations and related value **corrections** can be performed as well

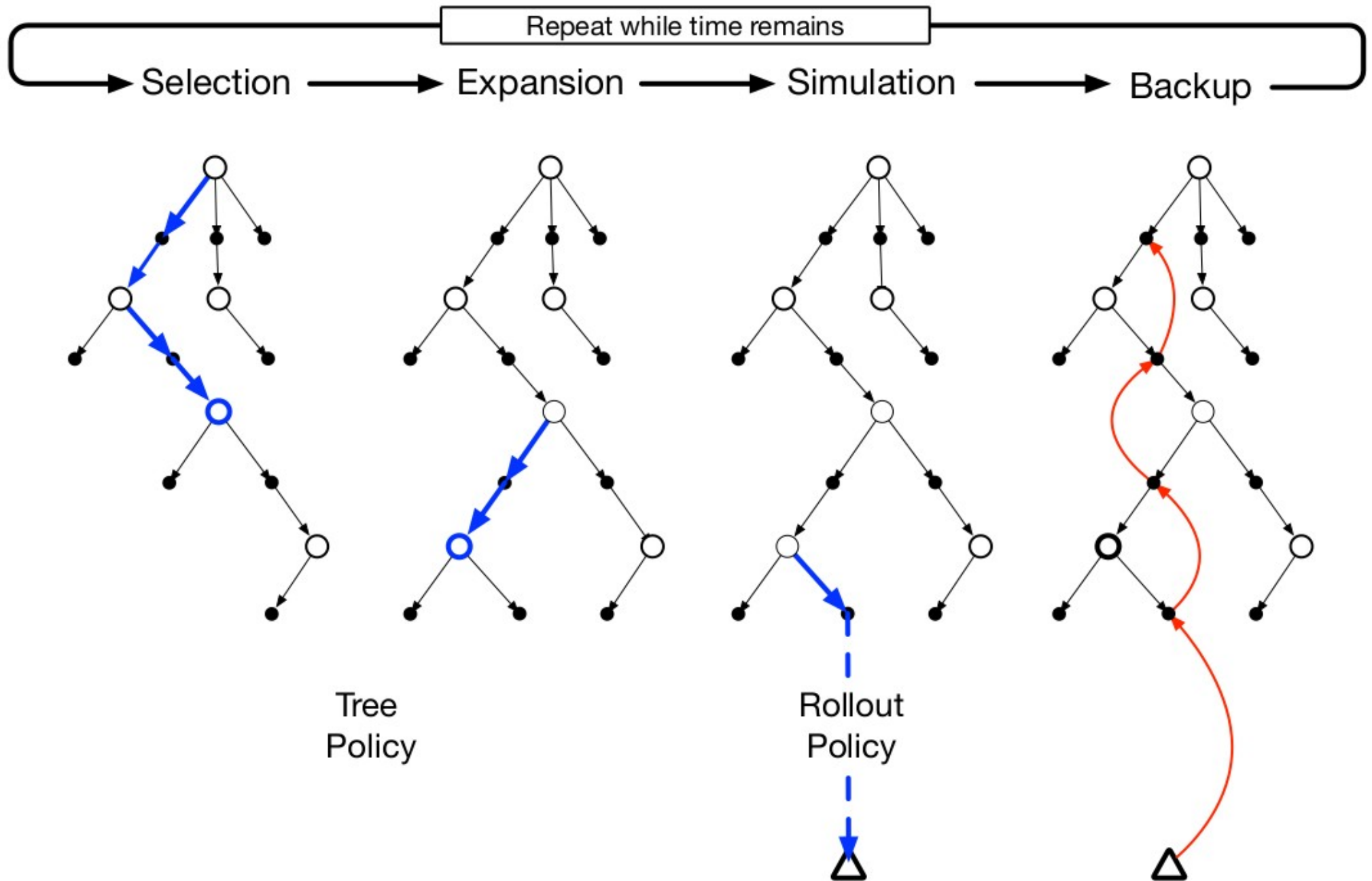
Monte Carlo Tree Search (MCTS)

- **MCTS** is a **recent** and strikingly **successful** example of **decision-time planning**
- **MCTS** is a **rollout algorithm** enhanced by a **strategy** to **direct future simulations** toward more **highly rewarding trajectories**
- **MCTS** is largely responsible for the improvement in computer Go from weak amateur (2005) to grandmaster level (2015)
- **MCTS** has proved to be **effective** in a **wide variety of competitive settings** including both **game playing** and **other domains** with **environment model simple enough for fast multistep simulations**
- The applications of MCTS to **real-world problems** is a current **challenge** of RL research.

Monte Carlo Tree Search (MCTS)

- **Core idea of MCTS:** successively focus multiple simulations starting at the current state by extending the initial portion of trajectories that have received high evaluations from earlier simulations
- For the most part, the actions in the simulated trajectories are generated using a simple **rollout policy**
- If the rollout policy and the model do not require a lot of **computation**, many simulated trajectories can be generated
- **State-Action values** are **estimated** as the **average** of the simulated returns from that state-action.
- **Value estimations** are stored **only** for the **subset of state-action pairs that are most likely to be reached in a few steps**. This forms a **tree** rooted in the current state

Monte Carlo Tree Search (MCTS)



Monte Carlo Tree Search (MCTS)

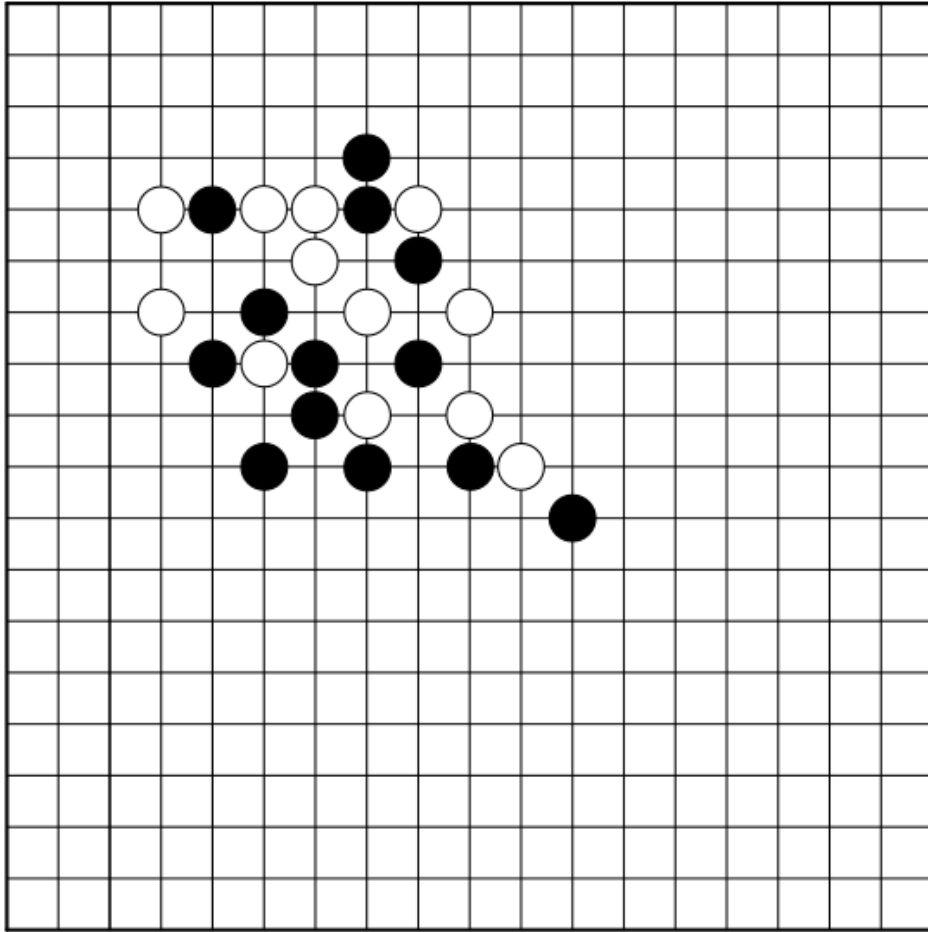
- **MCTS incrementally extends the tree** by adding new state-nodes that look **promising** based on the results of the previous trajectories
- **Outside the tree** the **rollout policy** is used to select actions
- **Inside the tree** something **better** can be done using action value estimates. We select actions according to a **tree policy** that uses the **UCB selection rule**

$$A_t \doteq \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

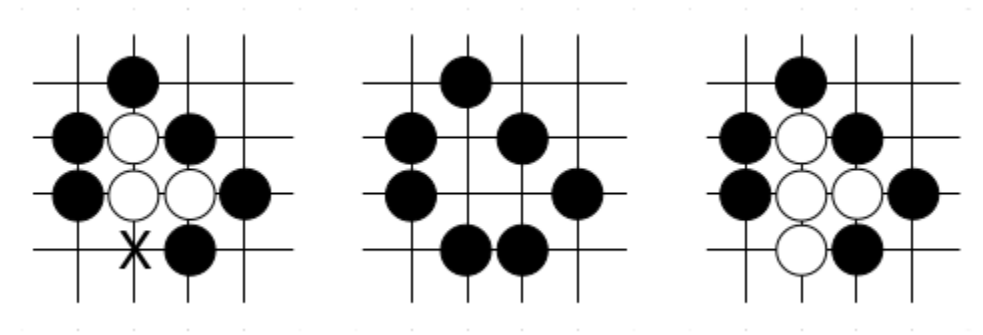
with

- t **total number of action-selections** performed so far
- $N_t(a)$ **number of times action** a **has been selected** prior to time t
- $c > 0$ controls the degree of **exploration**

AlphaGo



A Go board configuration



Go capturing rule

- It uses an improved version of MCTS called **Asynchronous policy and value MCTS (APV-MCTS)**

Main differences:

- **Tree expansion**: instead of using stored action values to select **unexplored edges** from leaf nodes, APV-MCTS uses **probabilities** supplied by a **13-layer deep convolutional ANN (SL-policy network)**
- **State node evaluation**: instead of using only the return of a rollout simulation performed by a **rollout policy**, APV-MCTS uses also a value function v_θ , another **13-layer deep convolutional ANN (value network)** computed by evaluating a policy network (**RL-policy network**) learned previously by reinforcement learning. The final value is:

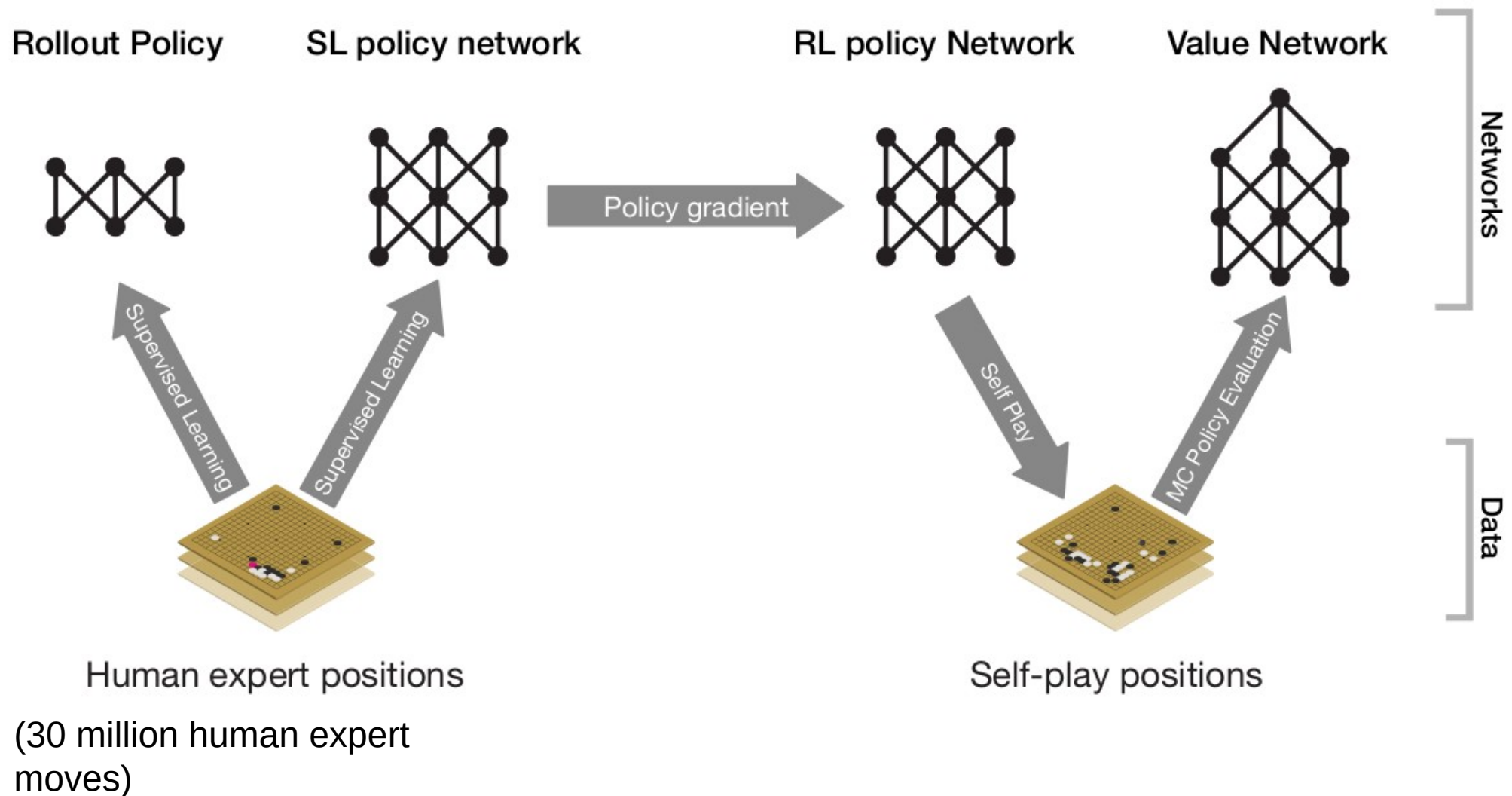
$$v(s) = (1 - \eta)v_\theta(s) + \eta G$$

↗ **Rollout policy**
(fast)

↘ **RL-policy network**
(high performance)

where G is the return of the rollout.

AlphaGo pipeline



Silver, D. et al. (2016). **Mastering the game of Go with deep neural networks and tree search**. Nature, 529(7587):484–489.

1. SL and RL policy networks

- Similar to REINFORCE network but use 13 convolutional layers
- **Output:** a softmax unit for each point of the 19x19 board
- **Input:** a 19 x 19 x 48 image stack in which each point of the Go board is represented by 48 binary/integer features (e.g., occupied or not)
- The **SL policy network is trained** by stochastic gradient ascent from human **expert data** (3 weeks on 50 processors, 57% accuracy)
- The **RL policy network is initialized to the SL policy network** and then **trained** by **policy gradient RL** over **simulated games** between the current RL policy network and earlier versions of the same network to prevent overfitting (it **wins 80%** games against the SL policy, **85%** games against MCTS with 100000 simulations).

2. Value network

- **Structure:** similar to **SL and RL policy networks**
- **Output:** single output unit
- **Input:** equal to the input of the **SL and RL policy networks** with an additional binary feature giving the **current color to play**
- **Training:** **MC policy evaluation** used to train the network from data obtained from a large number of **self-play games** played using the **RL policy** (some weeks on 50 GPUs)

3. Rollout policy

- **Structure:** very simple linear network trained by supervised learning from a corpus of 8 million human moves
- **Performance:** It provides reasonably accurate actions very **quickly**. The **SL or RL policy networks** are not used for rollout because they are **too slow** (1000 complete simulations per second on each processing thread)
- **Features:** can be **computed more quickly** than those used by SL/RL policy networks